

Smart Campus Parking and Traffic Management System

CSA0925 – Programming in Java
AWT/Swing + JDBC (MySQL)

PART 1 – PSEUDOCODE

1. Database Connection Module

```
BEGIN DBConnection
  CONSTANT URL <- "jdbc:mysql://localhost:3306/campus_parking"
  CONSTANT USER <- "root"
  VARIABLE conn <- NULL

  FUNCTION getConnection()
    IF conn is NULL OR conn is closed THEN
      LOAD com.mysql.cj.jdbc.Driver
      conn <- DriverManager.getConnection(URL, USER, PASSWORD)
    END IF
    RETURN conn
  END FUNCTION

  FUNCTION testConnection()
    TRY
      CALL getConnection()
      IF conn is valid THEN RETURN null // success
    CATCH SQLException
      RETURN error message
    END TRY
  END FUNCTION

  FUNCTION close()
    IF conn is not NULL THEN
      conn.close()
      conn <- NULL
    END IF
  END FUNCTION
END DBConnection
```

2. User Management Module

```
BEGIN UserDAO

  FUNCTION addUser(user)
    sql <- "INSERT INTO users(full_name,email,phone,
      user_type,id_number,address) VALUES(?,?,?,?,?,?)"
    ps <- PreparedStatement(sql)
    SET ps.param(1) <- user.fullName
    SET ps.param(2) <- user.email
    SET ps.param(3) <- user.phone
    SET ps.param(4) <- user.userType
    SET ps.param(5) <- user.idNumber
    SET ps.param(6) <- user.address
    EXECUTE ps
    RETURN rowsAffected > 0
  END FUNCTION

  FUNCTION getAllUsers()
    sql <- "SELECT * FROM users ORDER BY full_name"
    stmt <- Statement() // plain Statement for simple SELECT
    rs <- stmt.executeQuery(sql)
    list <- empty list
    WHILE rs.next()
      user <- mapRowToUser(rs)
      list.add(user)
    END WHILE
    RETURN list
  END FUNCTION
```

```

FUNCTION searchUsers(keyword)
  sql <- "SELECT * FROM users WHERE full_name LIKE ?
        OR email LIKE ? OR id_number LIKE ?"
  ps <- PreparedStatement(sql)
  SET all params <- "%" + keyword + "%"
  RETURN executeAndMap(ps)
END FUNCTION

FUNCTION updateUser(user)
  sql <- "UPDATE users SET full_name=?,email=?,phone=?,
        user_type=?,id_number=?,address=? WHERE user_id=?"
  ps <- PreparedStatement(sql)
  SET parameters from user object
  EXECUTE ps
  RETURN rowsAffected > 0
END FUNCTION

FUNCTION deleteUser(userId)
  sql <- "DELETE FROM users WHERE user_id=?"
  ps <- PreparedStatement(sql)
  SET ps.param(1) <- userId
  EXECUTE ps
  RETURN rowsAffected > 0
END FUNCTION

```

END UserDao

3. Vehicle Management Module

BEGIN VehicleDAO

```

FUNCTION addVehicle(vehicle)
  sql <- "INSERT INTO vehicles(user_id,license_plate,
        vehicle_type,make,model,color) VALUES(?,?,?,?,?,?)"
  ps <- PreparedStatement(sql)
  SET parameters from vehicle object
  EXECUTE ps
  RETURN rowsAffected > 0
END FUNCTION

FUNCTION getVehicleByPlate(plate)
  sql <- "SELECT v.*, u.full_name AS owner_name
        FROM vehicles v JOIN users u ON v.user_id=u.user_id
        WHERE v.license_plate=?"
  ps <- PreparedStatement(sql)
  SET ps.param(1) <- plate
  rs <- ps.executeQuery()
  IF rs.next() THEN RETURN mapRowToVehicle(rs)
  ELSE RETURN null
END FUNCTION

FUNCTION getVehiclesByUser(userId)
  sql <- "SELECT * FROM vehicles WHERE user_id=?"
  ps <- PreparedStatement(sql)
  SET ps.param(1) <- userId
  RETURN executeAndMap(ps)
END FUNCTION

```

END VehicleDAO

4. Parking Zone and Slot Module

```
BEGIN ParkingZoneDAO

    FUNCTION getAllZones()
        sql <- "SELECT z.*,
            (SELECT COUNT(*) FROM parking_slots s
             WHERE s.zone_id=z.zone_id
             AND s.status='Available') AS available_slots
            FROM parking_zones z ORDER BY z.zone_name"
        stmt <- Statement()
        RETURN executeAndMapZones(stmt, sql)
    END FUNCTION

    FUNCTION addZone(zone)
        VALIDATE zoneName is not empty
        VALIDATE hourlyRate is valid number
        sql <- "INSERT INTO parking_zones VALUES(?,?,?, ?, ?, ?)"
        ps <- PreparedStatement(sql)
        SET parameters from zone object
        EXECUTE ps
    END FUNCTION

END ParkingZoneDAO

BEGIN ParkingSlotDAO

    FUNCTION getAvailableSlots(zoneId)
        sql <- "SELECT s.*, z.zone_name FROM parking_slots s
            JOIN parking_zones z ON s.zone_id=z.zone_id
            WHERE s.zone_id=? AND s.status='Available'
            ORDER BY s.slot_number"
        ps <- PreparedStatement(sql)
        SET ps.param(1) <- zoneId
        RETURN executeAndMapSlots(ps)
    END FUNCTION

    FUNCTION updateSlotStatus(slotId, status)
        sql <- "UPDATE parking_slots SET status=? WHERE slot_id=?"
        ps <- PreparedStatement(sql)
        SET ps.param(1) <- status
        SET ps.param(2) <- slotId
        EXECUTE ps
    END FUNCTION

    FUNCTION getSlotsByStatus(status)
        sql <- "SELECT * FROM parking_slots WHERE status=?"
        ps <- PreparedStatement(sql)
        SET ps.param(1) <- status
        RETURN executeAndMapSlots(ps)
    END FUNCTION

END ParkingSlotDAO
```

5. Reservation Module

```
BEGIN ReservationDAO

    FUNCTION addReservation(reservation)
        // Step 1: Conflict check
        checkSql <- "SELECT COUNT(*) FROM reservations
            WHERE slot_id=? AND status='Active'
            AND NOT (reserved_until<=? OR reserved_from>=?)"
```

```

        ps <- PreparedStatement(checkSql)
        SET ps.param(1) <- reservation.slotId
        SET ps.param(2) <- reservation.reservedFrom
        SET ps.param(3) <- reservation.reservedUntil
        IF count > 0 THEN
            THROW SQLException("Conflicting reservation exists")
        END IF

        // Step 2: Insert
        sql <- "INSERT INTO reservations
                (user_id,vehicle_id,slot_id,
                 reserved_from,reserved_until,status)
                VALUES(?,?,?,?,'Active')"
        ps <- PreparedStatement(sql, RETURN_GENERATED_KEYS)
        SET parameters from reservation object
        EXECUTE ps

        // Step 3: Update slot
        CALL slotDAO.updateSlotStatus(slotId, 'Reserved')
        RETURN generatedKey
    END FUNCTION

    FUNCTION cancelReservation(reservationId)
        reservation <- getReservationById(reservationId)
        sql <- "UPDATE reservations SET status='Cancelled'
                WHERE reservation_id=?"
        ps <- PreparedStatement(sql)
        EXECUTE ps
        CALL slotDAO.updateSlotStatus(reservation.slotId, 'Available')
    END FUNCTION
END ReservationDAO

```

6. Vehicle Entry / Exit – Stored Procedures & CallableStatement

```

STORED PROCEDURE sp_vehicle_entry(vehicleId, slotId, userId) -> sessionId
    GET slotStatus FROM parking_slots WHERE slot_id = slotId
    IF slotStatus != 'Available' THEN
        RAISE ERROR "Slot is not available"
    ELSE
        INSERT INTO parking_sessions(vehicle_id, slot_id, user_id,
                                     entry_time, status)
            VALUES(vehicleId, slotId, userId, NOW(), 'Active')
        sessionId <- LAST_INSERT_ID()
        UPDATE parking_slots SET status='Occupied' WHERE slot_id=slotId
    END IF
END PROCEDURE

STORED PROCEDURE sp_vehicle_exit(sessionId) -> fee
    GET entryTime, slotId, hourlyRate
    FROM parking_sessions JOIN parking_slots JOIN parking_zones
    WHERE session_id = sessionId
    hours <- MAX( CEIL( minutes_diff(NOW, entryTime) / 60 ), 1 )
    fee <- hours x hourlyRate
    UPDATE parking_sessions SET exit_time=NOW(), duration_hrs=hours,
                                fee_amount=fee, status='Completed'
    UPDATE parking_slots SET status='Available' WHERE slot_id=slotId
    RETURN fee
END PROCEDURE

BEGIN SessionDAO

    FUNCTION recordEntry(vehicleId, slotId, userId)

```

```

        cs <- CallableStatement("{CALL sp_vehicle_entry(?,?,?,?)}")
        SET cs.IN(1) <- vehicleId
        SET cs.IN(2) <- slotId
        SET cs.IN(3) <- userId
        REGISTER cs.OUT(4) <- INTEGER // sessionId
        cs.execute()
        RETURN cs.getInt(4)
    END FUNCTION

    FUNCTION recordExit(sessionId)
        cs <- CallableStatement("{CALL sp_vehicle_exit(?,?)}")
        SET cs.IN(1) <- sessionId
        REGISTER cs.OUT(2) <- DECIMAL // fee
        cs.execute()
        RETURN cs.getBigDecimal(2)
    END FUNCTION

```

END SessionDAO

7. Parking Pass Module

BEGIN PassDAO

```

    FUNCTION addPass(pass)
        sql <- "INSERT INTO parking_passes
                (user_id,vehicle_id,zone_id,pass_type,
                 start_date,end_date,pass_fee,status)
                VALUES(?,?,?,?,?,?,?, 'Active')"
        ps <- PreparedStatement(sql, RETURN_GENERATED_KEYS)
        SET parameters from pass object
        EXECUTE ps
        RETURN generatedKey
    END FUNCTION

    FUNCTION expireOldPasses()
        sql <- "UPDATE parking_passes SET status='Expired'
                WHERE end_date < CURDATE() AND status='Active'"
        stmt <- Statement() // plain Statement for batch update
        stmt.executeUpdate(sql)
    END FUNCTION

    FUNCTION autoCalculateEndDate(startDate, passType)
        IF passType = 'Monthly' THEN RETURN startDate + 1 month
        IF passType = 'Semester' THEN RETURN startDate + 6 months
        IF passType = 'Annual' THEN RETURN startDate + 1 year
    END FUNCTION

```

END PassDAO

8. Payment Module

BEGIN PaymentDAO

```

    FUNCTION addPayment(payment)
        sql <- "INSERT INTO payments
                (user_id,session_id,pass_id,amount,payment_mode,status)
                VALUES(?,?,?,?,?,?)"
        ps <- PreparedStatement(sql, RETURN_GENERATED_KEYS)
        SET ps.param(1) <- payment.userId
        SET ps.param(2) <- payment.sessionId (or NULL)
        SET ps.param(3) <- payment.passId (or NULL)
        SET ps.param(4) <- payment.amount
        SET ps.param(5) <- payment.paymentMode
        SET ps.param(6) <- payment.status

```

```

        EXECUTE ps
        RETURN generatedKey
    END FUNCTION

    FUNCTION getAllPayments()
        sql <- "SELECT p.*, u.full_name FROM payments p
                JOIN users u ON p.user_id=u.user_id
                ORDER BY p.payment_date DESC"
        stmt <- Statement()
        RETURN executeAndMapPayments(stmt, sql)
    END FUNCTION

```

END PaymentDAO

9. Violation Module

BEGIN ViolationDAO

```

    FUNCTION addViolation(violation)
        VALIDATE vehicle is registered
        VALIDATE fineAmount > 0
        sql <- "INSERT INTO violations
                (vehicle_id,slot_id,violation_type,
                 description,fine_amount,status)
                VALUES(?,?,?,?,'Pending')"
        ps <- PreparedStatement(sql)
        SET parameters from violation object
        EXECUTE ps
        RETURN rowsAffected > 0
    END FUNCTION

    FUNCTION updateViolationStatus(violationId, status)
        sql <- "UPDATE violations SET status=?
                WHERE violation_id=?"
        ps <- PreparedStatement(sql)
        SET ps.param(1) <- status // 'Paid' or 'Waived'
        SET ps.param(2) <- violationId
        EXECUTE ps
    END FUNCTION

    FUNCTION searchViolations(keyword)
        sql <- "SELECT vl.*, v.license_plate, s.slot_number
                FROM violations vl
                JOIN vehicles v ON vl.vehicle_id=v.vehicle_id
                LEFT JOIN parking_slots s ON vl.slot_id=s.slot_id
                WHERE v.license_plate LIKE ?
                OR vl.violation_type LIKE ?"
        ps <- PreparedStatement(sql)
        SET params <- "%" + keyword + "%"
        RETURN executeAndMapViolations(ps)
    END FUNCTION

```

END ViolationDAO

10. Reports Module

BEGIN ReportsPanel

```

    FUNCTION zoneOccupancyReport()
        sql <- "SELECT zone name, zone type, total_slots,
                COUNT(CASE WHEN status='Available' THEN 1 END) AS available,
                COUNT(CASE WHEN status='Occupied' THEN 1 END) AS occupied
                FROM parking_zones JOIN parking_slots GROUP BY zone_id"
        EXECUTE via Statement
    END FUNCTION

```

```

        FILL table with results
        SHOW occupancy% = (occupied / total) x 100 in summary
    END FUNCTION

    FUNCTION revenueReport()
        sql <- "SELECT zone_name,
                    SUM(session fee) AS session_revenue,
                    SUM(pass fee) AS pass_revenue
                FROM zones JOIN sessions JOIN passes GROUP BY zone_id"
        EXECUTE via Statement
        SHOW total revenue in summary
    END FUNCTION

    FUNCTION dailyRevenueReport()
        sql <- "SELECT DATE(payment_date), COUNT(*), SUM(amount)
                FROM payments WHERE status='Paid'
                GROUP BY DATE(payment_date)
                ORDER BY date DESC LIMIT 30"
        EXECUTE via Statement
        FILL table with last 30 days data
    END FUNCTION

    FUNCTION utilizationReport()
        sql <- "SELECT zone_name, total_slots,
                    ROUND(occupied_count * 100.0 / total_slots) AS pct
                FROM parking_zones JOIN parking_slots
                GROUP BY zone_id ORDER BY pct DESC"
        EXECUTE via Statement
        DISPLAY utilization per zone
    END FUNCTION

END ReportsPanel

```

11. Main Application Flow

```

BEGIN Application

    FUNCTION main()
        SET UIManager <- SystemLookAndFeel
        CREATE MainFrame
        SHOW MainFrame
    END FUNCTION

BEGIN MainFrame

    ON STARTUP
        CREATE 7 panels:
            UserVehiclePanel, ZoneSlotPanel, ReservationPanel,
            EntryExitPanel, PassPaymentPanel, ViolationPanel, ReportsPanel
        BUILD JMenuBar: File | Manage | Reports | Database | Help
        ADD header (dark blue), JTabbedPane, status bar
        TEST DBConnection
            IF success -> status <- "Connected to campus_parking"
            IF failure -> SHOW warning dialog

    ON MENU "Refresh All"      -> CALL refresh() on all 7 panels
    ON MENU "Test Connection" -> CALL DBConnection.testConnection()
    ON MENU "Exit"
        SHOW "Exit application?" confirm dialog
        IF YES -> DBConnection.close() -> System.exit(0)

END MainFrame
END Application

```


PART 2 – IMPLEMENTATION AND OUTPUT

Technology Stack

Component	Technology Used
Language	Java 21
GUI Framework	Java Swing – JFrame, JTabbedPane, JTable, JComboBox, JMenuBar, GridBagLayout, JSplitPane
Database	MySQL 8.0
Connectivity	JDBC – Statement, PreparedStatement, CallableStatement
Driver	mysql-connector-j-8.3.0.jar
Build	javac (JDK 21) – 51 .class files generated

Project Structure

[illegible]

Database Schema – 9 Tables

users	(user_id PK, full_name, email UNIQUE, phone, user_type, id_number UNIQUE, address)
vehicles	(vehicle_id PK, user_id FK->users, license_plate UNIQUE, vehicle_type, make, model, color)
parking_zones	(zone_id PK, zone_name UNIQUE, zone_type, location, total_slots, hourly_rate, description)
parking_slots	(slot_id PK, zone_id FK->zones, slot_number, slot_type, status)
reservations	(reservation_id PK, user_id FK, vehicle_id FK, slot_id FK, reserved_from, reserved_until, status)
parking_sessions	(session_id PK, vehicle_id FK, slot_id FK, user_id FK, entry_time, exit_time, duration_hrs, fee_amount, status)
parking_passes	(pass_id PK, user_id FK, vehicle_id FK, zone_id FK, pass_type, start_date, end_date, pass_fee, status)
payments	(payment_id PK, user_id FK, session_id FK, pass_id FK, amount, payment_mode, payment_date, status)
violations	(violation_id PK, vehicle_id FK, slot_id FK, violation_type, description, fine_amount, violation_date, status)

JDBC Statement Types Used

Statement Type	Used In	Purpose
Statement	getAllUsers(), getAllZones(), Reports panels	Simple SELECT queries with no parameters
PreparedStatement	All INSERT, UPDATE, DELETE, filtered SELECT	Parameterized queries – prevents SQL injection
CallableStatement	SessionDAO.recordEntry(), SessionDAO.recordExit()	Invokes MySQL stored procedures with IN/OUT params

Input Validation Implemented

Scenario	Handling Method
Duplicate email / ID / license plate	MySQL UNIQUE constraint + SQLException caught and shown in dialog
Unavailable or reserved slot	Slot status checked before entry; stored procedure raises error if not Available
Invalid date format for reservations	SimpleDateFormat parse wrapped in try/catch; user shown correct format
Conflicting reservations	Overlap query using PreparedStatement checks before insert
Vehicle already parked	Active session lookup before recording new entry
Invalid numeric input (fee, rate)	BigDecimal parse wrapped in try/catch with

	error dialog
Database connection failure	Startup test with user-friendly warning dialog and fix instructions
SQL exceptions	All DAO methods declare/throw SQLException; UI catches and shows error

Key Features by Module

Module	Features Implemented
Users & Vehicles	Register, update, delete, search by keyword, filter by type
Zones & Slots	CRUD zones, add/edit slots, colour-coded grid (green/red/yellow/grey), status filter
Reservations	Conflict detection, datetime validation, active/cancel/expire status management
Entry / Exit	CallableStatement entry/exit, auto fee calc, active sessions table, history table
Passes & Payments	Monthly/Semester/Annual pass issuance, auto end-date, payment history tab
Violations	Record violation, search, mark Paid/Waived, colour-coded status table
Reports (8 types)	Zone occupancy, vehicle history, revenue, active sessions, utilisation %, violation summary, pass status, daily revenue

Output Screenshots

1. Main Application Window – header, 7 tabs, menu bar, status bar showing "Connected to campus_parking database"
2. Users & Vehicles Panel – user registration form, search bar, data table
3. Zones & Slots Panel – colour-coded slot grid (green=Available, red=Occupied, yellow=Reserved)
4. Reservations Panel – booking form with zone/slot dropdown and conflict detection
5. Entry / Exit Panel – active sessions table, fee calculation result dialog
6. Passes & Payments Panel – pass issuance form and payment history table
7. Violations Panel – violation recording form and colour-coded status table
8. Reports Panel – zone-wise occupancy report with summary statistics

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

UsersVehicles

User Details

Full Name*:

Email*:

Phone*:

ID Number*:

User Type:

Student

Address:

Add

Update

Delete

Clear

Search:

Go

Show All

ID	Name	Email	Phone	Type	ID Number	Address	Registered
1	ASDFGH	ASDFGH12@	23456789	Student	23	ASXDRCTFVGVBHJNLK	2026-09-02 02:18:26.0

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

UsersVehicles

Vehicle Details

Owner (User)*:

ASDFGH (Student)

License Plate*:

Vehicle Type:

Car

Make:

Model:

Color:

Add

Update

Delete

Clear

Search:

Go

Show All

ID	Owner	License Plate	Type	Make	Model	Color	Registered
----	-------	---------------	------	------	-------	-------	------------

Report generated: Zone-wise Occupancy

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

Parking Zones

Zone Name*:
Zone Type: Student
Location:
Total Slots: 10
Hourly Rate (₹): 0.00
Description:

Add ZoneUpdateDeleteClear

ID	Zone Name	Type	Location	Total	Available	Rate (₹)	Description
1	Zone-A	Student	Block A - North	50	10	10.00	Student parking near ho...
2	Zone-B	Faculty	Block B - Admin	30	10	0.00	Free faculty parking
3	Zone-C	Visitor	Main Gate	20	10	20.00	Visitor parking
4	Zone-D	Staff	Block D - East	40	10	5.00	Staff parking
5	Zone-E	Service	Service Entrance	10	10	0.00	Service vehicles only
6	Zone-F	General	Central Campus	60	10	15.00	General purpose parking

Parking Slots

Zone*: Zone-A (Student)
Slot Number*:
Slot Type: Regular
Status: Available
Filter Status: All

Add SlotUpdateDeleteClearFilter

ID	Zone	Slot No	Type	Status
6	Zone-A	A01	Handicapped	Available
12	Zone-A	A02	EV	Available
18	Zone-A	A03	Regular	Available
24	Zone-A	A04	Regular	Available
30	Zone-A	A05	Regular	Available
36	Zone-A	A06	Regular	Available
42	Zone-A	A07	Regular	Available
48	Zone-A	A08	Regular	Available
54	Zone-A	A09	Regular	Available
60	Zone-A	A10	Regular	Available

AvailableOccupiedReservedMaintenance

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

New Reservation

User*: ASDFGH (Student)
Zone*: Zone-A - Student
From (yyyy-MM-dd HH:mm)*: 2026-09-02 20:42
Available slots: 10

Vehicle*:
Slot*: A01 [Handicapped]
Until (yyyy-MM-dd HH:mm)*: 2026-09-02 21:42

Reserve SlotCancel SelectedRefreshFilter: All

ID	User	License Plate	Zone	Slot	From	Until	Status	Created
----	------	---------------	------	------	------	-------	--------	---------

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & Vehicles

Zones & Slots

Reservations

Entry / Exit

Passes & Payments

Violations

Reports

New Reservation

User*: ASDFGH (Student)
Zone*: Zone-A - Student
From (yyyy-MM-dd HH:mm)*: 2026-09-02 20:42
Available slots: 10

Vehicle*:
Slot*: A01 [Handicapped]
Until (yyyy-MM-dd HH:mm)*: 2026-09-02 21:42

Reserve SlotCancel SelectedRefreshFilter: All

ID	User	License Plate	Zone	Slot	From	Until	Status	Created
----	------	---------------	------	------	------	-------	--------	---------

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Vehicle Entry

License Plate*:

Zone*:

Zone-A (Student)

Available Slot*:

A01 [Handicapped]

Find Vehicle / Reload Slots

Record Entry

Vehicle Exit & Fee Collection

License Plate*:

Session: --

Fee: --

Payment Mode:

Cash

Lookup Active Session

Record Exit & Pay

Active Sessions

Session ID	Vehicle	User	Zone	Slot	Entry Time	Status
------------	---------	------	------	------	------------	--------

Parking History

Session ID	Vehicle	Zone	Slot	Entry	Exit	Hours	Fee (₹)	Status
------------	---------	------	------	-------	------	-------	---------	--------

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Parking PassesPayment History

Issue Parking Pass

User*:

ASDFGH (Student)

Vehicle*:

Zone*:

Zone-A

Pass Type:

Monthly

Start Date (yyyy-MM-dd)*:

2026-09-02

End Date (yyyy-MM-dd)*:

2026-10-02

Pass Fee (₹)*:

500.00

Payment Mode:

Cash

Issue Pass

Cancel Selected

Auto-Expire Passes

Pass ID

User

Vehicle

Zone

Type

Start

End

Fee (₹)

Status

Issued

Connected to campus_parking database.

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Record Violation

License Plate*:

Lookup

Violation Type*:

No Parking Permit

Description:

Fine Amount (₹)*:

500.00

Status: Pending

Add Violation

Mark Paid

Mark Waived

Delete

Search (plate / type):

Go

Show All

ID	License Plate	Slot	Type	Description	Fine (₹)	Date	Status
----	---------------	------	------	-------------	----------	------	--------

Connected to campus_parking database.

30°

Search

ENG IN 92% 08:42 PM 02-09-2026

Smart Campus Parking & Traffic Management System

FileManageReportsDatabaseHelp

Smart Campus Parking & Traffic Management System

Wed Sep 02 20:42:01 IST 2026

Users & VehiclesZones & SlotsReservationsEntry / ExitPasses & PaymentsViolationsReports

Generate Reports

Zone-wise Occupancy

Vehicle-wise History

Revenue Report

Active Sessions

Utilization %

Violation Summary

Pass Status Report

Daily Revenue

Report: Zone-wise Occupancy

Zone	Type	Total	Available	Occupied	Reserved	Maintenance
Zone-A	Student	50	10	0	0	0
Zone-B	Faculty	30	10	0	0	0
Zone-C	Visitor	20	10	0	0	0
Zone-D	Staff	40	10	0	0	0
Zone-E	Service	10	10	0	0	0
Zone-F	General	60	10	0	0	0

Summary
Total Slots: 210
Available: 60
Occupied: 0
Occupancy Rate: 0.0%

Report generated: Zone-wise Occupancy

30°

Search

ENG IN 92% 08:42 PM 02-09-2026

PART 3 – INDIVIDUAL CONTRIBUTION WRITE-UPS

Note: As per submission guidelines, register numbers and names are not included in this document. Each write-up below corresponds to one team member.

Team Member 1 – Database Design, Backend Connectivity and DAO Layer

This assignment involved building a complete Smart Campus Parking and Traffic Management System using Java Swing and JDBC with a MySQL backend. My contribution focused on the database design, JDBC connectivity, and the backend DAO layer of the project.

I designed the relational database schema with nine tables — users, vehicles, parking zones, parking slots, reservations, parking sessions, parking passes, payments, and violations — with all primary key and foreign key relationships carefully defined to maintain referential integrity. I wrote two stored procedures: `sp_vehicle_entry`, which atomically inserts a session record and marks the slot as Occupied, and `sp_vehicle_exit`, which calculates the parking fee based on duration and hourly rate and marks the slot as Available on exit.

I implemented the `DBConnection` singleton class managing the JDBC connection lifecycle, and developed six DAO classes — `UserDAO`, `VehicleDAO`, `ParkingZoneDAO`, `ParkingSlotDAO`, `SessionDAO`, and `PaymentDAO`. In these classes I used three JDBC statement types as required: plain `Statement` for simple `SELECT` all queries, `PreparedStatement` for all parameterized `INSERT`, `UPDATE`, `DELETE` and filtered `SELECT` operations to prevent SQL injection, and `CallableStatement` in `SessionDAO` to invoke the stored procedures with IN and OUT parameters for vehicle entry and exit.

I tested all SQL queries in MySQL Workbench before Java integration, debugged JDBC driver configuration issues, and verified that slot status transitions (Available → Occupied → Available) were atomic. This assignment gave me deep practical knowledge of relational database normalization, JDBC statement types, stored procedure design, and secure database programming.

Team Member 2 – GUI Design, Main Frame, User/Vehicle and Entry/Exit Panels

My contribution focused on designing and implementing the graphical user interface, specifically the `MainFrame`, `UserVehiclePanel`, and `EntryExitPanel`.

I built the `MainFrame` as the root application window using `BorderLayout`, with a dark blue header banner, a `JTabbedPane` containing seven functional tabs, a complete `JMenuBar` with five menus (File, Manage, Reports, Database, Help) including keyboard mnemonics and accelerators, and a status bar at the bottom. I also implemented the startup database connectivity test that shows a user-friendly warning dialog with fix instructions if MySQL is unavailable.

I developed the `UserVehiclePanel` with two sub-tabs using `GridBagLayout` for precise form alignment. The Users sub-tab provides full CRUD — add, update, delete, and search by keyword — with input validation for required fields and email format. The Vehicles sub-tab

links vehicles to registered owners via a dynamic JComboBox and validates the license plate uniqueness at the database level. Both tabs use JTable with row selection to pre-populate the edit form.

I built the EntryExitPanel, which is operationally the most critical module. The entry side looks up a vehicle by license plate, loads available slots for the selected zone, checks whether the vehicle is already parked, and invokes the stored procedure via CallableStatement. The exit side looks up the active session by plate, calls the exit stored procedure which calculates the fee, and records the payment automatically. I also implemented the active sessions and parking history tables that update in real time. This work gave me strong practical experience in event-driven Swing programming, layout management, and end-to-end workflow design.

Team Member 3 – Reservations, Passes, Violations, Zone/Slot Panel, Reports and Documentation

My role covered the reservation system, zone and slot management, parking passes, violations, the reporting module, and the project documentation.

I built the ZoneSlotPanel using a JSplitPane with GridBagLayout forms. The slot table uses a custom prepareRenderer to colour-code rows — green for Available, red for Occupied, yellow for Reserved, and grey for Maintenance — giving operators an immediate visual overview. I developed ParkingZoneDAO and ParkingSlotDAO including the getAvailableSlots and getSlotsByStatus methods used across multiple panels.

I developed the ReservationPanel with datetime validation, dynamic slot loading based on the selected zone, and conflict detection. The ReservationDAO uses a PreparedStatement overlap query to prevent double-booking on the same slot within the same time window, and immediately updates the slot status to Reserved on success. I built the PassPaymentPanel with auto end-date calculation using Java LocalDate — Monthly adds one month, Semester six months, Annual one year — and a payment history sub-tab. Each pass issuance triggers an automatic payment record linked via foreign key.

For violations, I implemented the ViolationPanel with record creation, status updates (Paid/Waived), deletion, and keyword search, with colour-coded table rows by status. I developed the ReportsPanel with eight report types — zone occupancy, vehicle history, revenue, active sessions, utilisation percentage, violation summary, pass status, and daily revenue — using complex SQL with GROUP BY, CASE expressions, subqueries, and aggregates.

Finally, I handled all project documentation: the pseudocode for all eleven modules, the README setup guide, the GitHub repository structure, and this submission document. I also performed integration testing, identifying and fixing the JComboBox type mismatch compilation error and the Windows button rendering issue. This project significantly

strengthened my skills in analytical SQL, Swing custom rendering, and full-stack Java application development.

SDG RELEVANCE

SDG 9 – Industry, Innovation and Infrastructure

The system digitises and automates manual parking management on a university campus, replacing paper-based methods with a structured, database-driven Java application. This directly contributes to building resilient and innovative campus infrastructure.

SDG 11 – Sustainable Cities and Communities

By providing real-time slot availability, reservation management, and utilisation reports, the system reduces vehicle congestion within the campus. Drivers no longer circle looking for parking, reducing fuel consumption and improving the campus environment.

SDG 13 – Climate Action

The system includes dedicated EV (Electric Vehicle) slot types within the parking infrastructure. By tracking and prioritising EV slots, the system encourages adoption of electric vehicles on campus, contributing to reduced carbon emissions.